

# Unsupervised Learning: K-Means Clustering

Lab Instructor: Sir Saad Ahmed

Student: Mehr Ali

```
In [10]: import math
import random

# A simple 2D dataset.
# We can clearly see two groups visually:
# (1,1), (1,2) vs (10,10), (10,11)
dataset = [
    [1, 1], [1, 2], [2, 1],
    [10, 10], [10, 11], [11, 10]
]

def euclidean_distance(point1, point2):
    total = 0
    for i in range(len(point1)):
        diff = point1[i] - point2[i]
        total = total + (diff * diff)
    return math.sqrt(total)
```

```
In [11]: def get_closest_centroid(point, centroids):
    # Start with a very large distance (infinity) so the first distance we
    # check will always be smaller
    min_dist = float('inf')
    closest_index = -1

    for i in range(len(centroids)):
        dist = euclidean_distance(point, centroids[i])
        if dist < min_dist:
            min_dist = dist
            closest_index = i

    return closest_index
```

```
In [12]: def update_centroids(clusters):
    new_centroids = []

    for cluster in clusters:
        # If a cluster randomly ended up empty, keep it empty to avoid divide-by-zero
        if len(cluster) == 0:
            continue

        num_points = len(cluster)
```

```
# Calculate the average across all dimensions (X, Y)
new_centroid = []
for dim in range(len(cluster[0])):
    dim_sum = 0
    for point in cluster:
        dim_sum += point[dim]
    new_centroid.append(dim_sum / num_points)

new_centroids.append(new_centroid)

return new_centroids
```

```
In [13]: def k_means(dataset, k, max_iterations=10):
# Step 1: Initialize random centroids
# Pick the first 'k' points as a starting point (simple approach)
centroids = dataset[:k]

for iteration in range(max_iterations):
    print(f"--- Iteration {iteration + 1} ---")

    # Create empty Lists for our k clusters
    clusters = []
    for _ in range(k):
        clusters.append([])

    # Step 2: Assign points
    for point in dataset:
        centroid_index = get_closest_centroid(point, centroids)
        clusters[centroid_index].append(point)

    # Step 3: Update centroids
    new_centroids = update_centroids(clusters)

    # Check if they stopped moving
    if new_centroids == centroids:
        print("Centroids stopped moving! Convergence reached.")
        break

    centroids = new_centroids
    print("Centroids updated to:", centroids)

    return centroids, clusters

# Run the algorithm
k = 2
final_centroids, final_clusters = k_means(dataset, k)

for i in range(len(final_clusters)):
    print(f"Cluster {i + 1} points:", final_clusters[i])
```

```

--- Iteration 1 ---
Centroids updated to: [[1.5, 1.0], [8.0, 8.25]]
--- Iteration 2 ---
Centroids updated to: [[1.3333333333333333, 1.3333333333333333], [10.333333333333334, 10.333333333333334]]
--- Iteration 3 ---
Centroids stopped moving! Convergence reached.
Cluster 1 points: [[1, 1], [1, 2], [2, 1]]
Cluster 2 points: [[10, 10], [10, 11], [11, 10]]

```

## Checkpoint 1: Manual Centroid Update

Blue cluster points are P(10, 20) and Q(20, 20).

- Mean X =  $(10 + 20) / 2 = 15$
- Mean Y =  $(20 + 20) / 2 = 20$

So, the new blue centroid is **(15, 20)**.

## Student Activity 1

### 1) Add a random middle point [5, 5]

We run K-Means again after adding [5, 5] to the dataset and check which cluster it joins.

```

In [14]: dataset_with_middle = [
          [1, 1], [1, 2], [2, 1],
          [10, 10], [10, 11], [11, 10],
          [5, 5]
        ]

middle_centroids, middle_clusters = k_means(dataset_with_middle, 2)

for i in range(len(middle_clusters)):
    print(f"Cluster {i + 1} points:", middle_clusters[i])

middle_cluster_index = -1
for i in range(len(middle_clusters)):
    if [5, 5] in middle_clusters[i]:
        middle_cluster_index = i

print("Point [5, 5] joined cluster index:", middle_cluster_index)

```

```

--- Iteration 1 ---
Centroids updated to: [[1.5, 1.0], [7.4, 7.6]]
--- Iteration 2 ---
Centroids updated to: [[1.3333333333333333, 1.3333333333333333], [9.0, 9.0]]
--- Iteration 3 ---
Centroids updated to: [[2.25, 2.25], [10.333333333333334, 10.333333333333334]]
--- Iteration 4 ---
Centroids stopped moving! Convergence reached.
Cluster 1 points: [[1, 1], [1, 2], [2, 1], [5, 5]]
Cluster 2 points: [[10, 10], [10, 11], [11, 10]]
Point [5, 5] joined cluster index: 0

```

## 2) Change K to 3

Now we run the same original dataset with  $K = 3$  and observe the clusters.

```

In [15]: centroids_k3, clusters_k3 = k_means(dataset, 3)

for i in range(len(clusters_k3)):
    print(f"Cluster {i + 1} points:", clusters_k3[i])

--- Iteration 1 ---
Centroids updated to: [[1.0, 1.0], [7.0, 7.666666666666667], [6.5, 5.5]]
--- Iteration 2 ---
Centroids updated to: [[1.3333333333333333, 1.3333333333333333], [10.333333333333334, 10.333333333333334]]
--- Iteration 3 ---
Centroids stopped moving! Convergence reached.
Cluster 1 points: [[1, 1], [1, 2], [2, 1]]
Cluster 2 points: [[10, 10], [10, 11], [11, 10]]
Cluster 3 points: []

```

For this dataset,  $K = 3$  creates an extra split in points. It works technically, but  $K = 2$  is more natural because the data has two clear groups.

## Task 1: Calculate the Error (SSE)

```

In [16]: def calculate_sse(clusters, centroids):
    total_error = 0
    # Loop through every cluster
    for i in range(len(clusters)):
        # Loop through every point in that cluster
        for point in clusters[i]:
            # Calculate the squared euclidean_distance between point and centroid
            error = euclidean_distance(point, centroids[i]) ** 2
            # Add it to total_error
            total_error += error
    return total_error

sse_value = calculate_sse(final_clusters, final_centroids)
print("SSE:", sse_value)

```

SSE: 2.6666666666666666

## Task 2: Predict New Data

```
In [17]: def predict(new_point, final_centroids):  
        # Return the index of the closest centroid  
        return get_closest_centroid(new_point, final_centroids)  
  
        print("Point [8, 8] belongs to cluster:", predict([8, 8], final_centroids))
```

Point [8, 8] belongs to cluster: 1

## Task 3: Challenge - Handling 3D Data

```
In [18]: dataset_3d = [  
        [1, 2, 5], [2, 1, 4], [1, 1, 6],  
        [10, 8, 9], [9, 10, 8], [11, 9, 10]  
        ]  
  
        centroids_3d, clusters_3d = k_means(dataset_3d, 2)  
  
        print("Final 3D centroids:", centroids_3d)  
        for i in range(len(clusters_3d)):  
            print(f"3D Cluster {i + 1} points:", clusters_3d[i])
```

--- Iteration 1 ---

Centroids updated to: [[6.4, 6.0, 7.6], [2.0, 1.0, 4.0]]

--- Iteration 2 ---

Centroids updated to: [[10.0, 9.0, 9.0], [1.3333333333333333, 1.3333333333333333, 5.0]]

--- Iteration 3 ---

Centroids stopped moving! Convergence reached.

Final 3D centroids: [[10.0, 9.0, 9.0], [1.3333333333333333, 1.3333333333333333, 5.0]]

3D Cluster 1 points: [[10, 8, 9], [9, 10, 8], [11, 9, 10]]

3D Cluster 2 points: [[1, 2, 5], [2, 1, 4], [1, 1, 6]]

## Reflection (Task 3)

The code worked without crashing. The reason is that `euclidean_distance` uses a loop over `range(len(point1))`, so it automatically handles all dimensions. The `update_centroids` function also loops over all dimensions using `range(len(cluster[0]))`, so it calculates mean values for 3D (and higher-dimensional) points in the same way.